

Message Passing Interface

Finding Prime Numbers

Task 1 Open MPI Code

Finding Prime Numbers

Task 1 Open MPI Code Implementation

Architectural Approach:

Broadcast Setup: The root process parses n and configuration settings, broadcasting them via `MPI_Bcast` to all ranks.

Local Prime Table: Every rank independently computes a small prime table up to $\sqrt{n}$ using the Sieve of Eratosthenes to perform trial division efficiently.

Independent Search: Ranks search assigned odd candidates ($k = 3 + 2c$) in parallel and accumulate results in local dynamic buffers.

Gather & Merge: Root uses `MPI_Gather` to collect buffer sizes, `MPI_Gatherv` to collect raw prime arrays, performs an m -way merge across pre-sorted segments, and writes the output file.

Task 1 Open MPI Code Parallel Partitioning Schemes

To handle the increasing computational cost of trial division for larger numbers, three distribution schemes were evaluated:

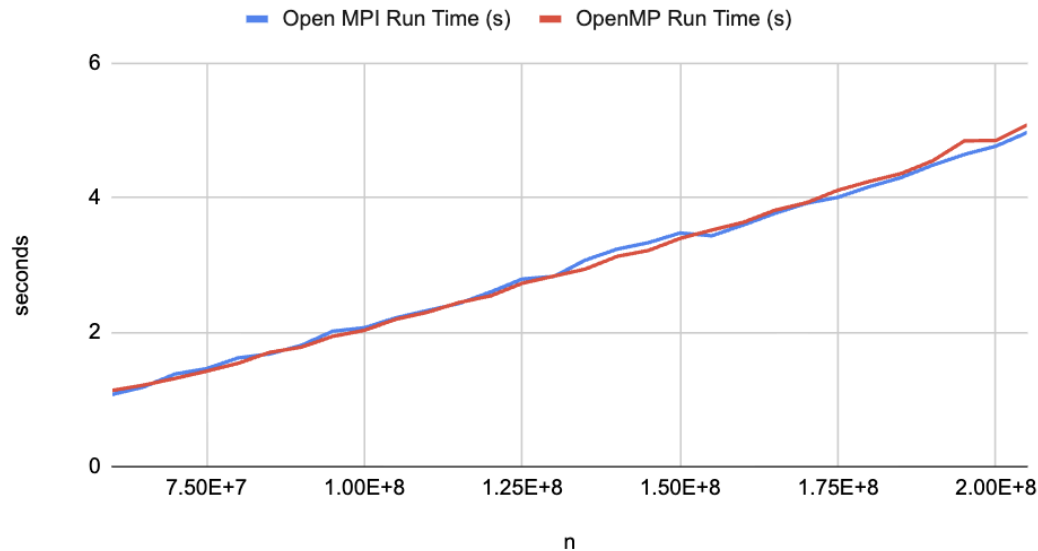
Block Partitioning (block): Splits candidates into equal chunks. Higher ranks process significantly larger numbers, leading to severe computational load imbalance and worker idling.

Cyclic Partitioning (cyclic): Distributes individual odd candidates round-robin ($c = r + i * p$). Interleaves heavy and light workloads evenly, achieving near-optimal load balance across all processes.

Block-Cyclic Partitioning (blockcyclic): Distributes configurable blocks of candidates (e.g. $s = 1024$) round-robin. Balances execution time while maintaining cache locality.

Task 1 Open MPI (graph 1)

Open MPI vs OpenMP Run Time (s) with increasing n

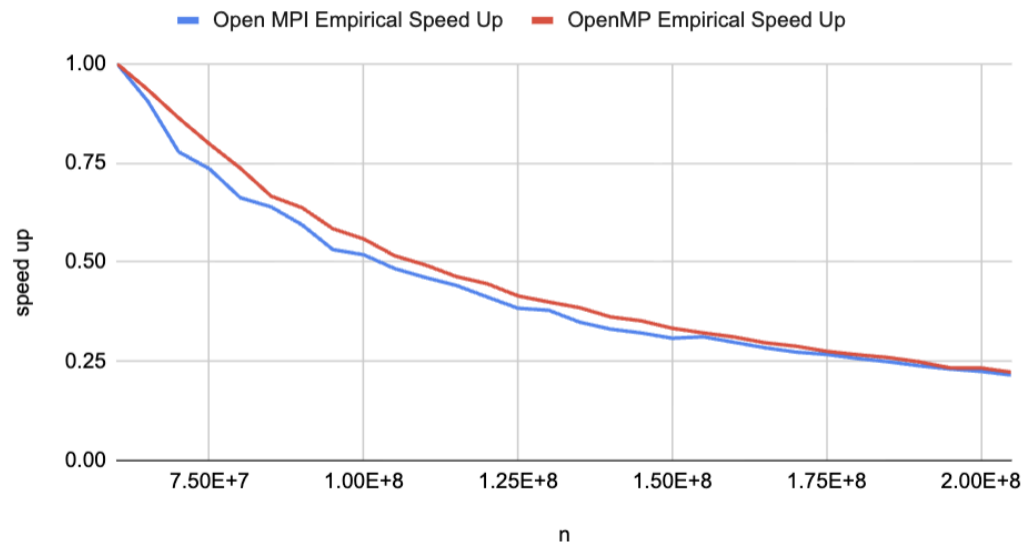


Open MPI was run with 4 processes and cyclic partitioning for this graph.

OpenMP was run with 4 threads for this graph.

Task 1 Open MPI (graph 2)

Open MPI vs OpenMP Empirical Speed Up with increasing n



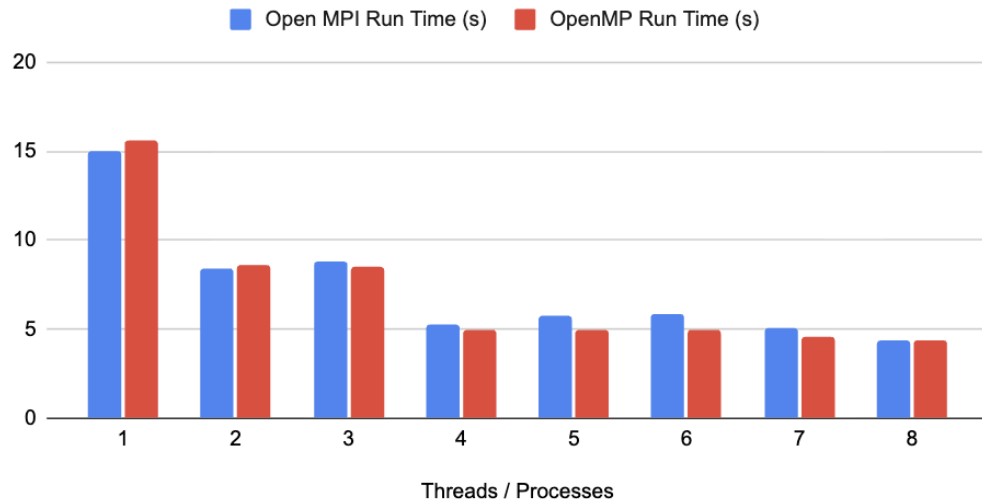
Open MPI was run with 4 processes and cyclic partitioning for this graph.

OpenMP was run with 4 threads for this graph.

Empirical speed up is calculated by dividing the base time by the current time at the current n

Task 1 Open MPI (graph 3)

Open MPI vs OpenMP Run Time (s) with increasing threads/processes



Open MPI was run with cyclic partitioning for this graph.

Both Open MPI and OpenMP was run with an n of 205,000,000 for this graph.

Task 2 Open MPI + Open MP Code

Finding Prime Numbers

Task 2 Open MPI + Open MP Code Implementation

Architectural Approach:

Two-Level Parallelism: Combines distributed memory parallelism (MPI processes across nodes) with shared memory parallelism (OpenMP threads per process).

Broadcast Setup: The root MPI process parses input parameters (n , distribution scheme, chunk size) and broadcasts them to all MPI ranks via `MPI_Bcast`.

Shared Small Prime Table: Each MPI process allocates a shared lookup table of primes up to $\sqrt{n}$ using the Sieve of Eratosthenes, accessible by all local OpenMP threads.

Thread-Level Accumulation: Threads evaluate assigned candidates in parallel, store local findings in thread-private buffers, and combine them into the process-level array.

Inter-Node Gather & Write: `MPI_Gather` and `MPI_Gatherv` aggregate results from all MPI processes to the root process, which merges pre-sorted segments and writes the final output to disk.

Task 2 Open MPI + Open MP Code Parallel Partitioning Schemes

Workload distribution operates across two distinct hierarchical layers to prevent thread idling and balance trial division workloads:

Layer 1: Process-Level Distribution (Inter-Process / MPI)

Block / Cyclic / Block-Cyclic Allocation: Candidates or candidate blocks are dealt round-robin among MPI processes based on their MPI rank r and process count P .

Coarse Load Balancing: Ensures high-level compute balance across distinct machines or CPU sockets.

Task 2 Open MPI + Open MP Code Parallel Partitioning Schemes

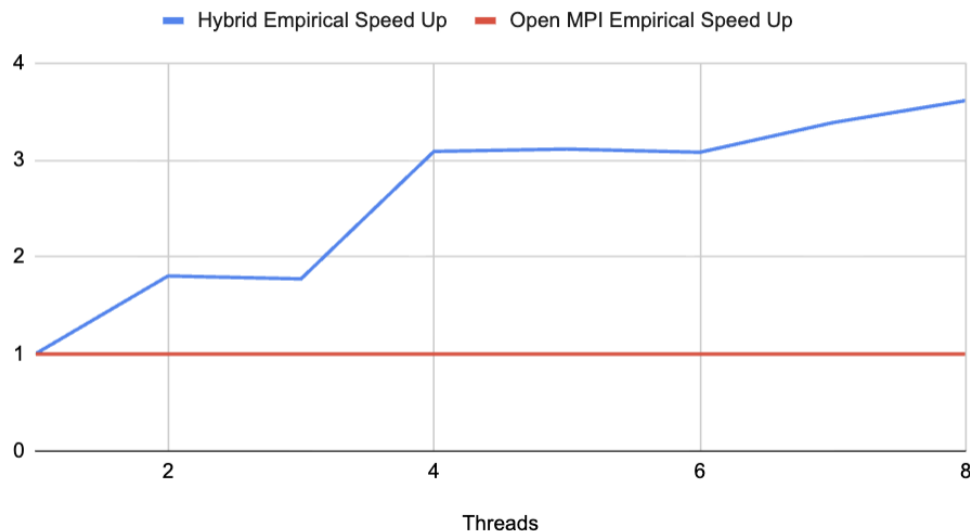
Layer 2: Thread-Level Distribution (Intra-Process / OpenMP)

Static / Dynamic / Guided Scheduling: OpenMP pragmas (`#pragma omp parallel for schedule(...)`) partition the process's assigned candidate chunk among local threads.

Fine-Grained Load Balancing: Because higher candidate values require more trial division steps, dynamic or cyclic thread scheduling prevents thread starvation within the same shared-memory node.

Task 2 Open MPI + Open MP (graph 1)

Hybrid vs Open MPI Empirical Speed Up by increasing threads

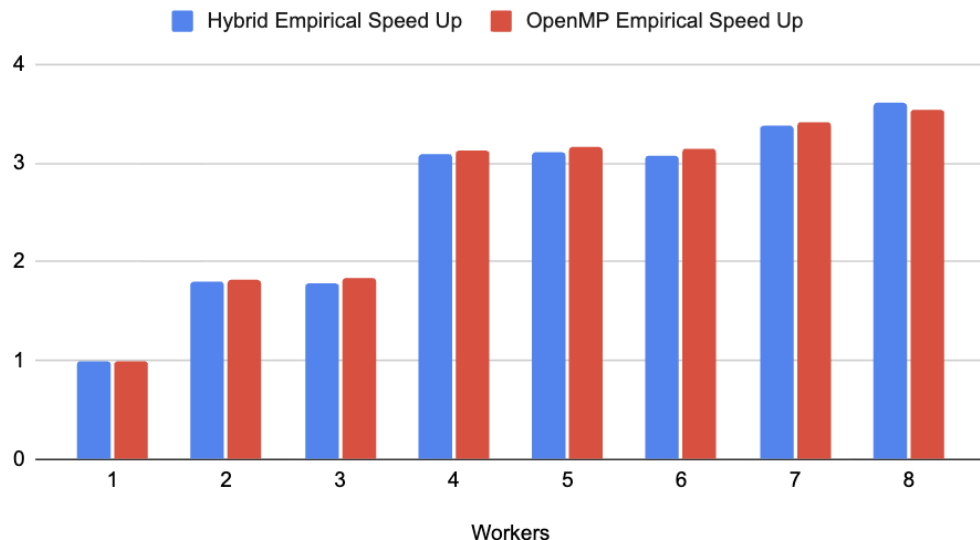


Both Hybrid and Open MPI was run with cyclic partitioning, 1 process and an n of 205,000,000 for this graph.

Empirical speed up is calculated by dividing the base time by the current time at the current n.

Task 2 Open MPI + Open MP (graph 2)

Hybrid vs OpenMP Empirical Speed Up



Hybrid was run with cyclic partitioning for this graph.

Both Hybrid and OpenMP were ran with an n of 205,000,000 for this graph.

Empirical speed up is calculated by dividing the base time by the current time at the current n .

Task 3 Performance Evaluation

Finding Prime Numbers

Task 3 Performance Evaluation

Empirical Speed-up for Task 1 against the serial version: 3.517167374

with n at 205,000,000 and processes at 8.

Empirical Speed-up for Task 2 against the serial version: 3.757205158

With n at 205,000,000 and total workers at 8.

There are 8 processors available on the machine that was used to test these numbers.

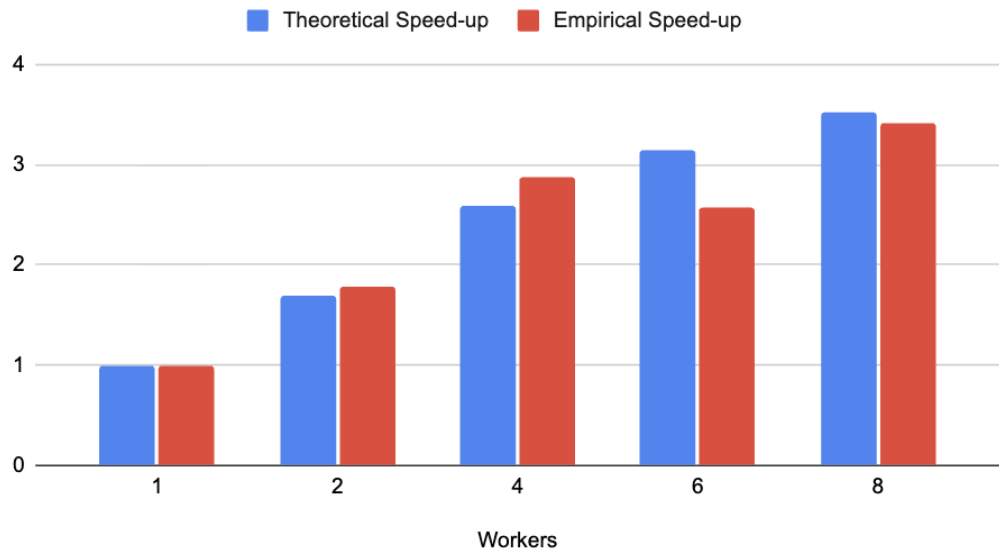
Task 3 Performance Evaluation

Workers	Theoretical Speed-up
1	1
2	1.69
4	2.59
6	3.14
8	3.52

This is the theoretical speed-up attained from computing using Amdahl's Law or Gustafson's Law.

Task 3 Performance Evaluation (graph 1)

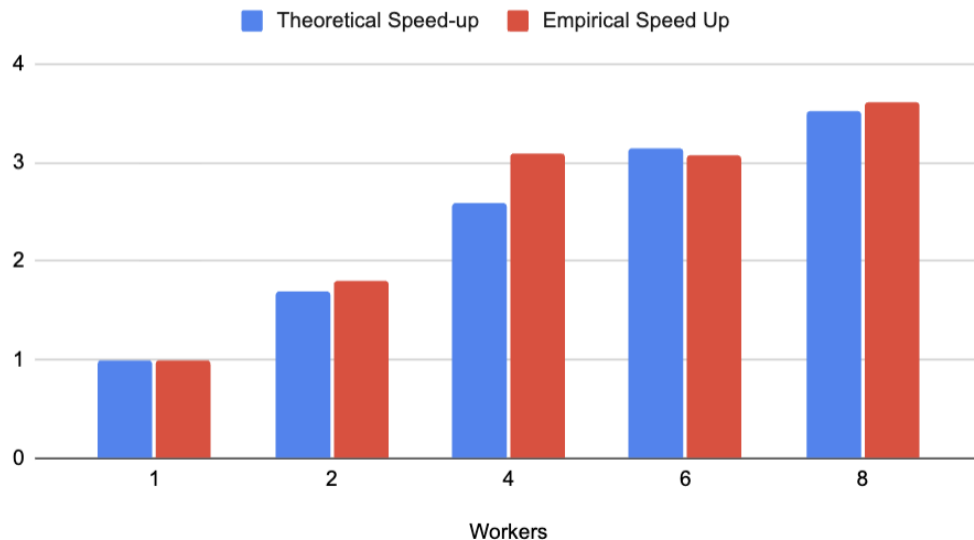
Task 1 Theoretical vs Empirical Speed-up



Open MPI (Task 1) was run with cyclic partitioning for this graph.

Task 3 Performance Evaluation (graph 2)

Task 2 Theoretical vs Empirical Speed Up



Hybrid (Task 2) was run with cyclic partitioning for this graph.

Q&A

Thank you for your attention